

Arquivos JSON

1. O que é JSON

JSON (*JavaScript Object Notation*) é um formato de texto usado para armazenar e trocar dados. Apesar do nome remeter ao JavaScript, o JSON é independente de linguagem: praticamente toda linguagem de programação moderna (Python incluído) sabe ler e gerar JSON, o que o torna um dos formatos mais usados para configuração de programas, troca de dados entre sistemas e resposta de APIs na internet.

Um arquivo (ou uma string) em formato JSON é escrito com uma notação muito parecida com a de um dicionário Python: pares `"chave": valor` entre chaves `{ }`, e listas entre colchetes `[ ]`. Por exemplo:

```
json

{
  "nome": "João da Silva",
  "idade": 30,
  "ativo": true,
  "telefones": ["123456789", "987654321"]
}
```

Python possui um módulo integrado (não precisa instalar nada) chamado `json`, usado para converter dados entre o formato Python e o formato JSON.

```
python

import json
```

2. Convertendo entre Python e JSON

O módulo `json` tem quatro funções principais, divididas em dois pares:

Função	Direção	Trabalha com
<code>json.dumps(objeto)</code>	Python → JSON	uma string
<code>json.loads(texto)</code>	JSON → Python	uma string
<code>json.dump(objeto, arquivo)</code>	Python → JSON	um arquivo
<code>json.load(arquivo)</code>	JSON → Python	um arquivo

Repare no padrão de nomes: as funções que terminam com **s** (de *string*) trabalham com strings na memória; as que não têm o **s** no final trabalham diretamente com um arquivo já aberto.

<code>dumps()</code>	<code>dump()</code>
Objeto -----> String JSON	Objeto -----> Arquivo JSON
<-----	<-----
<code>loads()</code>	<code>load()</code>

2.1 De Python para JSON — `dumps()` e `dump()`

```
python
import json

pessoa = {
    "name": "John",
    "age": 30,
    "married": True,
    "divorced": False,
    "children": ("Ann", "Billy"),
    "pets": None,
}

texto_json = json.dumps(pessoa)  # converte para uma string JSON
print(texto_json)
```

Saída:

```
{"name": "John", "age": 30, "married": true, "divorced": false, "children":
["Ann", "Billy"], "pets": null}
```

Ao converter, cada tipo Python vira o equivalente em JSON:

Python	JSON
<code>dict</code>	Object
<code>list</code> / <code>tuple</code>	Array
<code>str</code>	String
<code>int</code> / <code>float</code>	Number
<code>True</code>	<code>true</code>
<code>False</code>	<code>false</code>
<code>None</code>	<code>null</code>

Para **salvar direto em um arquivo**, em vez de gerar uma string, usa-se `json.dump()` (sem o "s"), passando o arquivo já aberto:

```
python

with open("arquivo.json", "w", encoding="utf-8") as arquivo:
    json.dump(pessoa, arquivo)
```

2.2 De JSON para Python — `loads()` e `load()`

```
python

import json

x = '{ "name": "John", "age": 30, "city": "New York"}'
y = json.loads(x)          # converte a string JSON em um dicionário Python

print(y["age"])            # 30
```

Para **ler de um arquivo**, usa-se `json.load()` (sem o "s"):

```
python

with open("arquivo.json", "r", encoding="utf-8") as arquivo:
    dados = json.load(arquivo)

print(dados["telefones"])
```

O resultado de `json.load()`/`json.loads()` é sempre uma estrutura Python "nativa" — geralmente um `dict` (se o JSON começa com `{`) ou uma `list` (se começa com `[`) — podendo conter outros dicionários e listas aninhados dentro dela, exatamente como acontece com dicionários e listas comuns em Python.

2.3 Formatando e ordenando o resultado (`indent` e `sort_keys`)

Por padrão, `json.dumps()`/`json.dump()` geram o JSON todo em uma única linha, o que dificulta a leitura. O parâmetro `indent` define quantos espaços usar de recuo em cada nível, deixando o resultado legível:

```
python

print(json.dumps(pessoa, indent=4))
```

Já o parâmetro `sort_keys=True` ordena as chaves em ordem alfabética no resultado — útil para deixar a saída mais previsível:

```
python

print(json.dumps(pessoa, indent=4, sort_keys=True))
```

2.4 Acentos e caracteres especiais — `ensure_ascii`

Por padrão, `json.dump()`/`json.dumps()` convertem qualquer caractere fora do padrão ASCII (como acentos: "ção", "é", "ã") em uma sequência de escape (ex.: `"\u00e7\u00e3o"`), que é tecnicamente válida, mas difícil de ler diretamente no arquivo. Para manter os acentos legíveis no arquivo gerado, usa-se `ensure_ascii=False`:

```
python

json.dump(dados, arquivo, indent=4, ensure_ascii=False)
```

A partir daqui, sempre que o material ou o projeto salvar um JSON com texto em português, o parâmetro `ensure_ascii=False` será usado por padrão.

3. Exemplo resolvido 1 — salvando um dicionário em um arquivo JSON

Considere o dicionário de estoque de uma papelaria:

```
python
```

```
import json

estoque = {
    "Caneta": {"marca": None, "preco": 10.0, "quantidade": 50},
    "Caderno": {"marca": "Tilibra", "preco": 20.0, "quantidade": 30},
    "Borracha": {"marca": "Faber-Castell", "preco": 5.0, "quantidade": 40},
}

# dump - converte uma estrutura Python para JSON e salva em um arquivo
with open("estoque.json", "w", encoding="utf-8") as arquivo:
    json.dump(estoque, arquivo, indent=4, ensure_ascii=False)
```

O arquivo `estoque.json` gerado fica assim:

```
json
{
  "Caneta": {
    "marca": null,
    "preco": 10.0,
    "quantidade": 50
  },
  "Caderno": {
    "marca": "Tilibra",
    "preco": 20.0,
    "quantidade": 30
  },
  "Borracha": {
    "marca": "Faber-Castell",
    "preco": 5.0,
    "quantidade": 40
  }
}
```

Repare que `None` virou `null`, e que a indentação deixou a estrutura bem parecida com a de um dicionário Python aninhado — pois é exatamente disso que se trata.

4. Exemplo resolvido 2 — lendo um arquivo JSON com dados aninhados

Considere um arquivo `heroes.json` com uma equipe de super-heróis, em que cada herói tem uma lista de poderes:

```
json
```

```
{
  "squadName": "Guardians of Justice",
  "members": [
    {
      "name": "Captain Valor",
      "age": 35,
      "powers": ["Super strength", "Flight", "Invulnerability"]
    },
    {
      "name": "Mystic Shadow",
      "age": 27,
      "powers": ["Shadow manipulation", "Invisibility", "Telepathy"]
    }
  ]
}
```

Para ler o arquivo e exibir apenas os nomes dos heróis:

```
python

import json

# load - carrega um arquivo JSON e converte para uma estrutura Python
with open("heroes.json", "r", encoding="utf-8") as arquivo:
    dicionario = json.load(arquivo)

# acessa a lista de heróis dentro do dicionário
herois = dicionario["members"]

# percorre a lista e exibe os nomes dos heróis
for item in herois:
    print(item["name"])
```

O ponto-chave deste exemplo é que, depois do `json.load()`, `dicionario` é um dicionário Python comum. `dicionario["members"]` é uma **lista de dicionários**, e cada um desses dicionários tem uma **lista de poderes** (`item["powers"]`) — a mesma lógica de acesso a estruturas aninhadas já usada em aulas anteriores com listas e dicionários, agora aplicada a dados vindos de um arquivo JSON.

Para saber mais: o formato JSON também aparece bastante fora de arquivos locais — é o formato mais comum usado por APIs na internet para trocar dados. Por exemplo, ao fazer uma requisição HTTP com a biblioteca externa `requests`, o método `.json()` da resposta já converte automaticamente o texto JSON recebido em um dicionário Python,

usando por baixo dos panos o mesmo `json.loads()` visto nesta aula. Esse é apenas um exemplo de uso mais avançado — não faz parte do conteúdo obrigatório desta aula.

5. Tema complementar: verificando se um arquivo existe com `os.path.exists()`

Na Aula 09, o tratamento de um arquivo ausente foi feito com `try/except FileNotFoundError`. Existe também uma outra forma de lidar com essa situação: **verificar antes** se o arquivo existe, usando a função `exists()` do módulo `os.path` (parte da biblioteca padrão do Python, no submódulo `os.path`).

```
python

import os

if os.path.exists("dados.json"):
    print("O arquivo existe!")
else:
    print("O arquivo ainda não foi criado.")
```

`os.path.exists(caminho)` retorna `True` se o arquivo (ou pasta) indicado existir, e `False` caso contrário — permitindo decidir o que fazer *antes* de tentar abrir o arquivo, em vez de tentar abrir e tratar o erro depois:

```
python

import json
import os

def carregar_configuracoes(caminho: str = "config.json") -> dict[str, object]:
    """Carrega um dicionário de configurações a partir de um arquivo JSON.

    Args:
        caminho: caminho do arquivo JSON de configurações.

    Returns:
        O dicionário de configurações lido do arquivo, ou um dicionário
        vazio caso o arquivo não exista.
    """
    if os.path.exists(caminho):
        with open(caminho, "r", encoding="utf-8") as arquivo:
            return json.load(arquivo)
    return {}
```

As duas abordagens (`try/except` e `os.path.exists()`) são válidas; `os.path.exists()` costuma deixar o código um pouco mais direto quando a única preocupação é "o arquivo

existe ou não", enquanto `try/except` continua sendo melhor quando é preciso tratar outros tipos de erro além da ausência do arquivo (permissão negada, arquivo corrompido etc.).

Exercícios complementares (tema: `os.path.exists()`)

C1. Implemente a função `carregar_configuracoes`, mostrada acima, que usa `os.path.exists()` para verificar se um arquivo JSON de configurações existe antes de tentar abri-lo; se não existir, a função deve devolver um dicionário vazio.

C2. Implemente uma função `salvar_backup(dados, caminho)` que recebe uma estrutura de dados e um caminho de arquivo. Antes de salvar, a função deve usar `os.path.exists()` para verificar se já existe um backup anterior nesse caminho; se existir, deve imprimir um aviso informando que o arquivo será sobrescrito. Em seguida, deve salvar `dados` no arquivo em formato JSON (com `indent=4` e `ensure_ascii=False`).

6. Exercícios principais (dificuldade crescente)

Exercício 1

Considere que o arquivo `notas.txt` apresenta uma listagem com os dados dos alunos de uma turma. Cada linha do arquivo apresenta os dados de um aluno, no formato:

```
RM, NOME, NOTA1, NOTA2, NOTA3, NOTA4
```

Implemente um programa que leia esse arquivo e gere um novo arquivo JSON no formato:

```
json
{
  "2101254": {
    "nome": "Benício Pires",
    "notas": [3.6, 10.0, 8.5, 7.8]
  },
  "2101624": {
    "nome": "Bruna Goncalves",
    "notas": [9.5, 8.0, 6.0, 5.5]
  }
}
```

Exercício 2

Considere o arquivo `foods.txt`, com três colunas, no formato abaixo, em que cada linha representa uma pessoa com suas informações:

```
NAME, ID, FAVORITEFOOD
```

Implemente um programa que, a partir do arquivo indicado, gere um arquivo JSON no formato:

```
json

{
  "1": {"name": "John Doe", "food": "pizza"},
  "2": {"name": "Jane Smith", "food": "sushi"},
  "3": {"name": "Michael Johnson", "food": "steak"}
}
```

Exercício 3

Considere o arquivo (`heroes.json`), com informações sobre uma equipe de super-heróis. Faça um programa em Python que leia o arquivo JSON e informe os nomes dos heróis que possuem o poder de voar (`"Flight"`).

Exercício 4

Implemente um sistema para cadastro de Pets, com as opções Inserir, Excluir e Listar Todos. Utilize um arquivo JSON para armazenar as informações. O arquivo JSON deve ter a estrutura abaixo e, conforme as operações forem realizadas, o arquivo deve ser modificado:

```
json

[
  {"tipo": "Cachorro", "nome": "Bebel", "idade": 4},
  {"tipo": "Hamster", "nome": "Pimpão", "idade": 2},
  {"tipo": "Cavalo", "nome": "Trovão", "idade": 6}
]
```

Dica: este exercício combina bem com o tema complementar desta aula

(`os.path.exists()`) — use-o para verificar, antes de cada operação, se o arquivo (`pets.json`) já existe.

7. Projeto de fundo — Enunciado da Aula 10

O e-commerce já sabe exportar seu catálogo para um arquivo de texto simples (Aula 09). Agora, evolua o projeto para que ele também saiba trabalhar com o formato **JSON**, mais estruturado:

1. Crie uma função (`exportar_catálogo_para_json`) que grave o catálogo inteiro em um arquivo (`.json`), preservando todos os campos de cada produto (nome, preço, estoque e tags). Como o tipo (`set`) não existe em JSON, as tags devem ser convertidas para uma lista (ordenada, por organização) antes da exportação.

2. Crie uma função `importar_catalogo_de_json` que leia esse arquivo JSON e reconstrua o catálogo, convertendo a lista de tags de volta para um `set`. Utilize `os.path.exists()` (tema complementar desta aula) para verificar se o arquivo existe antes de tentar abri-lo.

Nenhuma função das aulas anteriores deve ser alterada — assim como nas Aulas 08 e 09, a evolução deve ser **aditiva**.